Playing Wordle with Uppaal Stratego

Peter G. Jensen, Kim G. Larsen, Marius Mikučionis

Computer Science, Aalborg University, Denmark
{pgj,kgl,marius}@cs.aau.dk

Abstract. In this paper we model and solve the popular game Wordle using Uppaal Stratego. We model three different game-modes in terms of POMDPs, with more than 12,000 controllable actions. These constitute by far the largest models ever presented to Uppaal Stratego. Our experimental evaluation is encouraging: e.g. in the hard game-mode the partitioning-refinement learning method of Uppaal Stratego reduces the expected number of guesses from a baseline of 7.67 to 4.40 using 1 million training episodes. To better understand the convergence properties of our learning method we also study reduced versions of Wordle.

1 Introduction

Frits Vaandrager has for many years been one of the most faithful Uppaal users. Having in 1994 verified the tolerance on clock accuracy of the Philips Audio protocol using linear hybrid automata [6], Frits became aware of the new tool Uppaal (first release in 1995) and sent his Master thesis student David Griffioen to Aalborg in order to apply Uppaal to the automatic verification of an extended version with two senders and bus collision [5]. Since then Frits Vaandrager has been using Uppaal in several applications:

- In 2006 [11] Frits and co-authors used Uppaal to model and analyze formally model parts of Zeroconf, a protocol for dynamic configuration of IPv4 link-local addresses that has been defined in RFC 3927 of the IETF.
- Also in 2006 [27] Frits and co-authors modelled and analysed the biphase mark protocol – a convention for representing both a string of bits and clock edges in square waves. Uppaal was used to derive maximal tolerances on clock rates, for different instances of the protocol, and to support the general parametric verification carried out using the proof assistant PVS.
- For a case-study of a wafer scanner from the semiconductor industry Frits and co-authors used Uppaal in combination with the symbolic model checker SMV to compute (1) a simple yet optimal deadlock avoidance policy, and (2) an infinite schedule that optimizes throughput [14].
- Within the European project QUASIMODO Frits and co-authors [25] developed a detailed timed automata model of the clock synchronization algorithm being used in a wireless sensor network under development by the Dutch company Chess. Using Uppaal Frits discovered that in certain cases a static, fully synchronized network may eventually become unsynchronized

if the current algorithm is used, even in a setting with infinitesimal clock drifts.
- Using UPPAAL in combination with colored Petri nets Frits and co-authors modelled a challenging industrial case involving an existing state-of-the-art image processing pipeline within professional digital document printers [17,18]. The modelling effort was used to derive schedules for multiple concurrent jobs in the presence of limited resources (CPUs, memory, USB bandwidth, ..).

Besides applying UPPAAL, Frits Vaandrager has also been actively involved in development of the tool over the years. Here we mention:

- In 2003, Frits contributed to a prototype extension of UPPAAL with symmetry reduction [13]. For several examples the prototype demonstrated drastic reduction both in computation time and memory usage.
- Around 2000 Frits was coordinator of the European project AMETIST, with participation of several Dutch, French, German and Danish partners. During the project a strong request was made by the involved participants from control theory (more specifically by Professor Sebastian Engel from Dortmund): in addition to constraints and optimization with respect time they were even more interested in optimization with respect to energy. This lead to the extension of *priced* timed automata, and very quickly highly efficient implementation in UPPAAL CORA [3].
- In 2000 Frits, with Thomas Hune and Gerd Behrmann from Aalborg, developed, implemented and evaluated the a distributed verification engine for timed automata [4].
- In 2001, Frits, with Thomas Hune, Judi Romijn and Marielle Stoelinga, developed an extension of UPPAAL capable of synthesizing linear constraints for the correctness of parametric timed automata [16].

Moreover, Frits has been using UPPAAL in his teaching. Here we mention:

- with Roelof Hamberg, Frits Vaandrager have used the UPPAAL model checker in an introductory course on operating systems for first-year computer science students at the Radboud University Nijmegen. Using UPPAAL, their students have found mistakes in purported solutions to concurrency-control problems presented by Allen Downey in his popular textbook *The Little Book of Semaphores [9].*
- The unpublished note by Frits "A first Introduction to UPPAAL" [26] has for several years been part of the material used in first lectures on UPPAAL at Aalborg University. The note also shows that Frits has an affinity to games – in particular the note introduces the puzzle of "Gossiping Girls", which is regularly used as a mini-project exercise.

Since the early days of UPPAAL, considerable effort is now invested in the development of the branch UPPAAL STRATEGO, which uses combination of symbolic and machine learning techniques for generating safe and near-optimal

strategies from Timed or even Hybrid Markov Decision processes. In this paper, we celebrate Frits Vaandrager's 60'th birthday by describing how the extremely popular game WORDLE may be modelled and solved using UPPAAL STRATEGO. We hope that he will find enjoyment in this and become as an enthusiastic user of UPPAAL STRATEGO as he has been of UPPAAL.

The outline of the paper is as follows: in Section 2 we give a brief description of WORDLE. In Section 3 we give a short presentation of UPPAAL STRATEGO and its partition-based Q-learning method based on the well-known Monty Hall Problem. We also present an extended 4-door version of this problem illustrating the need for memory-full strategies. In Section 4 we model three different game-modes of WORDLE, including the so-called *hard mode*. As POMDPs the models are orders of magnitude larger than any other model previously presented to UPPAAL STRATEGO. The models have more than 12,000 controllable actions – i.e. all the 5-letter English words – to be selected based on observation of a `knowledge` memorizing data-structure. Finally in Section 5 we apply UPPAAL STRATEGO as a solver for WORDLE, while studying the impact of (combination of) game-mode(s), training budget and the impact of our partitioning-refinement technique. Our results are encouraging: e.g. for the hard game-mode UPPAAL STRATEGO reduces the expected number of guesses from a baseline of 7.67[1] to 4.40 after 1 million training episodes. For the largely unrestricted permissive game-mode, UPPAAL STRATEGO also makes a significant reduction from 14.33 expected guesses to 10.55. This is still far away from the maximum 6 guesses allowed in WORDLE, so a substantial increase of the training budget is required. To better understand the convergence of our UPPAAL STRATEGO encodings we also consider reduced version of WORDLE. E.g. reduction by a factor of 128 makes all game-modes converge to an average of 2.21 guesses with the permissive mode starting from a baseline of 12.78 guesses. In all the experiments the use of our partition-refinement based Q-learning method clearly demonstrates its advantage over a traditional, fully explicit Q-learning approach.

2 WORDLE

WORDLE is a web-based word game created and developed by Welsh software engineer Josh Wardle, and owned and published by The New York Times Company since 2022 [28]. The game has gained a large amount of popularity after Wardle added the ability for players to copy their daily results as emoji squares around December 2021, which were widely shared on Twitter. Many clones and variations of the game were also created, as were versions in languages besides English.

In WORDLE a Player has six attempts to guess a five-letter word selected by an Opponent. After each guess the Opponent will provide, with feedback in the form of colored tiles indicating when letters match or occupy the correct position.

[1] i.e. when choosing legal words uniformly random

More precisely, as illustrated in Fig. 1 after each guess, the letters which are not in the word are highlighted in grey, the letters which are in the word, but are in the wrong place are highlighted in yellow, and the letters which are in the correct spot in the word are highlighted green. If a guess contains repeated letters, but there is only one instance of that letter in the word, the second instance of the letter is marked grey.

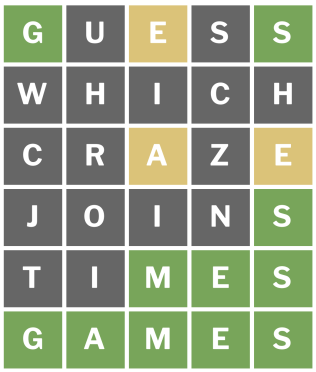


Fig. 1: WORDLE screenshot.

Now the guess/feedback-interaction between the Player and the Opponent may be viewed as a decision tree, e.g. Fig. 2 is part of the decision tree in the setting of just 7 words. In the figure only a single guess is shown at each node, but the full decision tree contains all (up to) 7 choices along with their respective sub-trees.

More formally, WORDLE is a *game* between the Opponent and the Player. In one setting, the Opponent is *antagonistic* and will non-deterministically select one of the 12,972 legal 5-letter English words in order to maximize the minimum number of guesses required by the Player. In the second setting – to be pursued in this paper – the Opponent uses a probability distribution to select the 5-letter word to be guessed. Here the objective of the Player is to find a strategy that will minimize the expected number of guesses needed to reveal the selected word. In both cases, the word selected by the Opponent is obviously not visible to the Player. In the probabilistic case the game at hand is thus formally a POMDP (Partially observable Markov decision process). In the following we shall demonstrate how the POMPD of WORDLE may be easily expressed using the rich modelling formalism of UPPAAL STRATEGO, and subsequently used for learning and analysis of a (near-) optimal strategy for the Player.

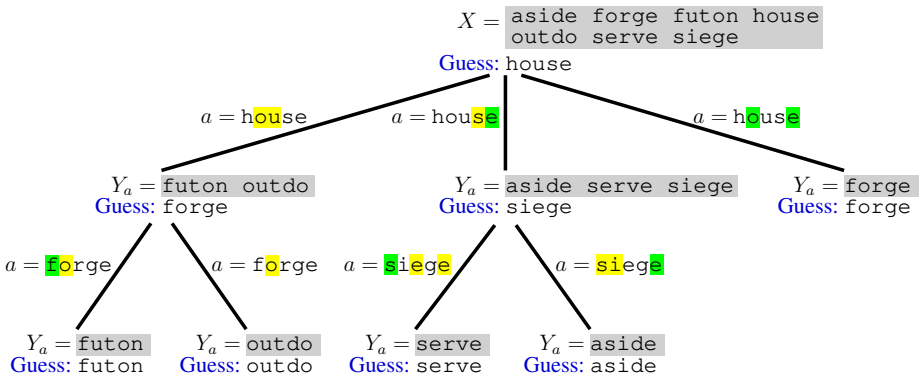


Fig. 2: (Partial) Decision Tree for Wordle (with 7 words).

Research Related to WORDLE Internet and popular press is buzzing with many speculations about the best opening guesses [2][3][4] and algorithms to minimize maximum and average number of guesses [5][6]. However the bad news is that WORDLE has been shown to be *NP-hard* [23] and *NP-complete* [24] and therefore require a lot of resources to compute absolute optimal solutions.

Reinforcement learning has been applied to find optimal human strategy using maximum correct letter probabilities [2]. They implement the game in Python and explore various score functions to come up with optimal 1-,2-,3-,4-,5-guess sequences out of 2,315 curated words to help the human to play more efficiently.

The game has been implemented in Matlab toolbox and has been applied to evaluate the probability of winning when employing 6 strategies including *active learning* with hard mode and knowledge [7].

3 UPPAAL STRATEGO

UPPAAL STRATEGO [8] is the newest branch of the UPPAAL Tool Suite using symbolic model checking and reinforcement learning to obtain safe and (near-optimal) strategies for Markov Decision Processes (MDP). The MPDs considered are infinite state, being based on either timed automata [1] (Timed MDPs) or hybrid automata [15] (Continuous-Space MDPs). Using symbolic techniques from model checking – and abstracting a given MDP $\mathcal{M}$ into a two-player timed game – a most permissive strategy σ_S for a given safety objective S is synthesized. Now, applying various versions of reinforcement learning [19], (still safe) substrategies σ_O optimizing a given optimization criteria O are obtained. Finally, UPPAAL STRATEGO may provide obtained strategies in terms of explainable decision trees, and furthermore supports synthesis of strategies taking partial observability into account, so-called POMDPS. So far UPPAAL STRATEGO has been applied successfully within a number of application domains including heating systems [20], adaptive cruise control [22], maneuvers of train in railway stations [21], swarm robotics [21], storm-water detention ponds [12], and traffic control [10].

Monty Hall Problem & Strategies To illustrate the use of UPPAAL STRATEGO for learning optimal strategies of POMDPs we consider a variant of the well-known Monty Hall Problem. Here a host ("Monty") provides a player with three doors, one containing a valuable prize and the other two containing a "gag", valueless prize. The contestant is offered a choice of one of the doors without knowledge of the content behind them. "Monty", who knows which door has the

[2] `https://www.3blue1brown.com/lessons/wordle2`
[3] `https://jonathanolson.net/experiments/optimal-wordle-solutions`
[4] `http://sonorouschocolate.com/notes/index.php/The_best_strategies_for_Wordle`
[5] `https://www.poirrier.ca/notes/wordle-optimal/`
[6] `https://github.com/TylerGlaiel/wordlebot`

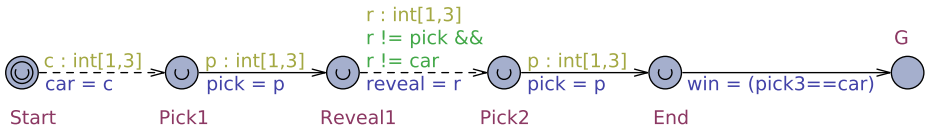


Fig. 3: Uppaal Stratego model for the original 3-door Monty Hall Problem.

prize, opens a door that the player did not select that has a gag prize, and then offers the player the option to switch from their choice to the other remaining unopened door. The probability problem arises from asking if the player should switch to the unrevealed door.

In Fig. 3 an MDP model of the original Monty Hall Problem in Uppaal Stratego is given. The MDP has 6 locations, with the choices of the host indicated by dashed edges and the choices by the player indicated by full edges. In `Start` the host (uniformly) randomly selects the door behind which the `car` is placed. In `Pick1` the player makes the initial choice of a door. In `Reveal` the host reveals opens a door not selected by the player and without the car. Now in `Pick2` the player makes the final choice of a door which, which in `End` will determine the value of `win`. The objective is to now to find a strategy `sp` of the player that will maximize the expected value of `win` when reaching `Hall.G`. Clearly the strategy will not be allowed to observe the value of the variable `car`, effectively making the model a POMDP.

In Uppaal Stratego synthesis of optimal strategies under partial observability is specified using optimization queries of the following form:

```
strategy Name = minE(Cost) [Bound]
            { ExprList1 } -> { ExprList2 } : <> Goal
```

where:

- `Name` is the name associated with the strategy, which can be later used to refer to the strategy when saving it to a file or examining its properties in other SMC queries.
- `minE(Cost)` asks the tool to minimize the expected cost defined by `Cost` expression. One can replace this entry with `maxE(Gain)` to maximize the expected gain defined by `Gain` expression.
- `Bound` defines a length of a stochastic simulation run, which can be described in terms of number of discrete transitions, absolute time or a limit on some variable in general.
- `Goal` is a state predicate defining the goal state, which also terminates the stochastic simulation run and marks it as “winning”.
- `ExprList1` and `ExprList2` are comma-separated expressions over the locations and variables defining the parts of the system state to be observed by the strategy.

Now, UPPAAL STRATEGO uses the Q-learning algorithm from [19] to learn an approximator for the function $\mathcal{Q}^*$, that for any value (E_1, E_2) of the expression-lists `ExprList1` and `ExprList2` and for any decision action a returns the minimal expected cost $\mathcal{Q}^*(E_1, E_2)(a)$ of reaching the `Goal` from (E_1, E_2) by choosing a. More precisely, for each decision action a and value E_1 of `ExprList1` the algorithm maintains a cost-function approximator of $\lambda E_2.\mathcal{Q}^*(E_1, E_2)(a)$ mapping values E_2 of `ExprList2` to the expected cost $\mathcal{Q}^*(E_1, E_2)(a)$. This approximator itself is maintained by a partition refinement method. Experience has indicated that it is preferable that the set of `ExprList1` values are discrete and relatively small (e.g. integer expressions, Boolean predicates over variables, distinct floating point values), where even a small change in value requires a different cost function altogether. In contrast, the set of `ExprList2` values can be large, represented by integers, clocks and floating point expressions, whenever small changes in value have small effects on cost. We shall later see the effect of placing expressions to be observed in either `ExprList1` or `ExprList2` in our experience with learning strategies for WORDLE.

Now returning to the Monty Hall Problem we pose the following query in UPPAAL STRATEGO:

```
strategy sp = maxE(win) [<=1]
      { Hall.location, pick, reveal } -> {}: <> Hall.G
```

The set-annotation `Hall.location, pick, reveal` restricts the strategy of the player to only depend on the locations of `Hall` and the values of `pick` and `reveal`. In particular, the value of the variable of `car` is not observable. Now, the value of the learned strategy `sp` is obtained by the following query:

```
E[<=1;1000] (max:win) under sp
```

returning 0.641 ± 0.029783 as a 95% confidence interval after 1000 runs. Further examination of the strategy reveals that in `sp` the player always opt for switching from the initial choice to the remaining unopened door.

A part of the computed strategy is seen in Listing 1.1. In the observed vector `(3,3,1)`, corresponding to being in `Pick2` having `pick=3` and `reveal=1`, we can see that action 2 has the highest computed Q-value of 0.7044. Here 2 denotes the action to switch and 0.7044 is an estimator of the expected win-ratio under the strategy. Conversely we can see that UPPAAL STRATEGO has learned that picking 1 after 1 was revealed by the host comes with zero reward.

4-door Monty Hall Problem & Memory-full Strategies In general optimal strategies for POMDPs require memory. This was not required in the simple POMDP of the Monty Hall Problem. To address the need for memory-full strategies and see how this can be handled in UPPAAL STRATEGO we consider in Fig. 4(a) an extended version of the Monty Hall Problem now with 4 doors, and an extra exchange between the host and the player. In all the moves of the player (full edges), the player may pick any of the four doors. The door selected in the

Listing 1.1: A snippet of the computed strategy for the 3-door Monty Hall Problem when in the observed state (`Pick2`, `pick=3`, `reveal=1`). Action indexes have been re-indexed for readability.

```
{...,"regressors":
        "(3,3,1)": {...,"regressor": {
                "3" : 0.3690...,
                "2" : 0.7044...,
                "1" : 0}, ...
            }
}
```

final pick `Pick3` is the final choice of the player. As for the host, the first door revealed in `Reveal1` is (as in the 3-door version) a randomly chosen door with no car and different from the first door selected by the player. In the second round, the door revealed by the host in `Reveal2` is randomly chosen among the doors with no car, and different from the second pick of the player, and different from the first door reveal by the host. Now the learning query of Uppaal Stratego

```
strategy sp4 = maxE(win) [<=1]
    { Hall.location, pick, reveal } -> {}: <> Hall.G
```

gives a strategy `sp4` with expected winning probability 66%. Examining the learned strategy it is found (perhaps not surprisingly) that the final picked door must be different from the second door picked as well as different from the last door revealed by the host.

However in the above strategy `sp4` is memoryless. In particular, in the last round the player is not allowed to take into account knowledge from the first round, e.g. what was the first door picked and the first door revealed. To allow for the strategy of the player to take information from all the past moves into account, Fig. 4(b) presents an extended model with round-numbered versions of the variables `pick` and `reveal` having been added. Clearly a principle that may be added to all acyclic POMDPs. Now the learning query:

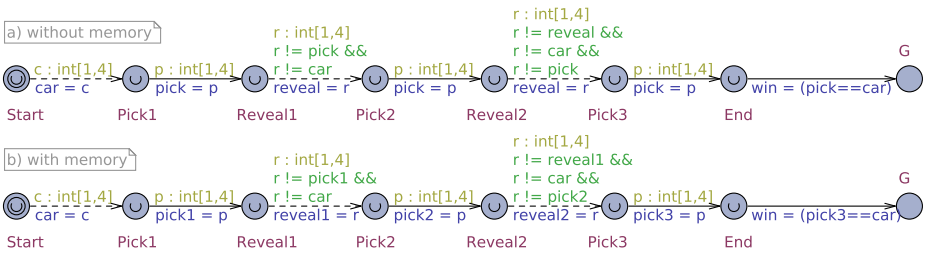


Fig. 4: Models for 4-door Monty Hall Problem: without memory and with.

```
strategy sp4h = maxE(win) [<=1] {
        Hall.location, pick1, reveal1, pick2, reveal2
    } -> {}: <> Hall.G
```

will learn a strategy for the player that may depend on the value of `pick` and `reveal` from all rounds, but of course still not the position of the `car`. Now the query `E[<=1;1000] (max:win) under sp4h` returns 0.744 ± 0.0270956 as a 95% confidence interval for the expected probability of winning under `sp4h`, a significant improvement compared to `sp4` that did not use memory. Finally, the probability estimation query:

```
Pr[<=1](<> Hall.G
    && !(pick3==pick2) && !(pick3==pick1)
    && (pick2==pick1) && !(pick3==reveal1)
    && !(pick3==reveal2) && !(pick2==reveal1)) under sp4h
```

returns ≈ 1, revealing interesting information about the strategy `sp4h`, e.g. that the door picked in the first and second round of the player should be *the same*, and that the door picked in the final round should be *different* from that.

We leave it to the interested reader (including Frits Vaandrager) to further investigate optimal strategies of N-door generalizations of the Monty Hall Problem using UPPAAL STRATEGO.

4 WORDLE in UPPAAL STRATEGO

Figure 5 shows the UPPAAL STRATEGO model of the POMDP for the WORDLE puzzle, where the *Player* repeatedly makes (controllable) decisions against a stochastic *Opponent*. The model follows the flow of the normal WORDLE game – although with several book-keeping steps of the Player allowing us to experiment with different game-modes such as the WORDLE hard-mode. Each word in the dictionary of valid words receives an index ranging from 0 to 12,972, captured by the constant `NWORDS`, of which the first 2,315 words can appear as solutions, captured by the constant `CURATED`. For convenience, WORDLE is extended to a timed-game in which each guess takes exactly one time-unit (tracked by the clock `t`) allowing us to track the total progression of time using the clock `guesses` which is never reset. Here is the general flow of the game:

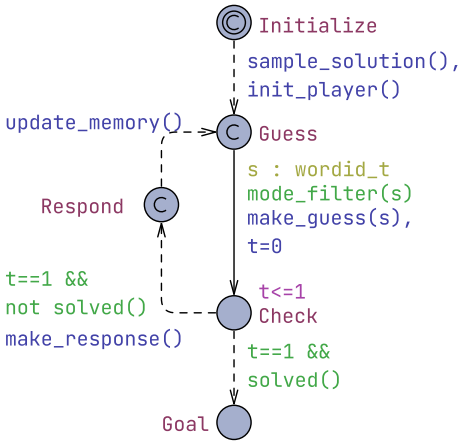


Fig. 5: UPPAAL STRATEGO model of WORDLE where a global clock `guesses` (not shown) keeps track of the total elapsed time, effectively measuring the number of guesses.

In `Initialize` location the Opponent must select a word from the curated solution-set by picking a word-index 0 and `CURATED`, this is done by the `sample_solution()` function. Meanwhile the `init_player()` function initializes the book-keeping memory of the player.

In `Guess` the Player may select the next guess `s : wordid_t`, with `wordid_t` defined as `int[0,NWORDS]`, that is valid with the current game-mode (checked by `mode_filter(s)`), like hard-mode which limits the guesses according to the specified game mode (e.g. hard-mode). The call to `make_guess(s)` effectively stores the guess of the player. Notice that this is the only location where the Player has decision (denoted by solid edges) whereas all other edges are under the control of the Opponent.

`Check` forces the Opponent to either validate that the guess is correct (checked by `solved()`) or that the guess was incorrect (`not solved()`), in which case a response is computed (`make_response()`). Notice that the invariant `t<=1` forces the opponent to reside in `Check` for at most 1 time unit, while the guards on the outgoing edges (`t==1`) enforces delay of exactly one time unit per guess.

`Goal` is the final location, denoting that the word proposed initially by the Opponent was found by the Player.

`Respond` allows for the Player to update the internal book-keeping memory.

The objective of the game is to reach `Goal` with the minimum expected value of the `guesses` clock.

While the `sample_solution()`, `init_player()`, `make_guess(s)` and `solved()` functions are straight forward, in that they simply copy, check and clear arrays, the `make_response()`, `update_memory()` and `mode_filter(s)` are of a more complex nature. We will explain the latter two in the following subsections as they are tied with the specific game mode employed. Let us first explain the `make_response()` in Listing 1.2.

At first glance, the response computation seems straight forward: respond `GREEN` on correctly placed letters, `YELLOW` if the letter appears in a different position, and `GRAY` otherwise. However, the *actual* feedback computation of Wordle is more subtle due to potential duplicate letters. For example, the word `hello` contains two 'l's. If the solution word is `afoul` (with a single 'l' present), then only a single letter 'l' is marked `YELLOW` and another 'l' is `GRAY`. Thus the response is computed in three passes: 1) to determine the number of occurrences of the different letters in the solution, 2) to mark `GREEN` responses (and deduct spent occurrences), and 3) to mark an adequate amount of characters as `YELLOW`.

With the main game flow in place, we can now discuss various game modes.

Game-Modes and Memory We experiment with three different game modes, that is, limitations for the Player guesses implemented by the `mode_filter(s)` guard in accordance with the (condensed) history of the guesses and responses:

1. *Hard mode*, where any prior hints of yellow and green must be respected in subsequent guesses,

Listing 1.2: Computes `response` (hint) based on `guess` and `solution`.

```
void make_response() {
    int[0,NPOS] counts[letter_t];
    // mark correct positions and count unmarked letters:
    for (p : pos_t)
        if (guess[p] == solution[p])
            response[p] = GREEN;
        else {
            response[p] = GRAY;
            ++counts[solution[p]];
        }
    // mark out of place letters:
    for (p : pos_t) {
        if (response[p] != GREEN) {
            if (counts[guess[p]] > 0) {
                response[p] = YELLOW;
                --counts[guess[p]];
            }
        }
    }
}
```

2. *Conservative mode*, where any prior hints of gray (i.e. non-existence of letter in the solution) must be respected s.t. any subsequent guess is delimited from using such letters, and
3. *Permissive mode*, where any new guess must change the internal knowledge structure.

The model uses a notion of memory to enforce the rules of the game mode which generic across all three modes and is represented by the following two data structures:

1. `knowledge`: an integer array of $|$`pos_t`$| \times |$`letter_t`$|$ size (i.e. 5×26) taking the value `MAYBE` (default) for "no knowledge", `SURELY_NOT` for `GRAY` hints (implying the letter can not be present at the given position) and `SURELY` for `GREEN` hints (implying that the letter must be present at the position), and
2. `counts`: an integer array of $|$`letter_t`$|$ size (`0`-initialized) to keep track of the maximal number of occurrences for each letter observed in a single word (e.g. getting `YELLOW` and `GREEN` hint on the `l`'s of `hello` would set the 12th position to 2).

Listing 1.3 shows `update_memory()` procedure to update the memory according to the rules above and the chosen `STRATEGY_TYPE`: conservative strategy (value `STRATEGY_PERMISSIVE`) or the permissive or combined strategy (values `STRATEGY_PERMISSIVE` and `STRATEGY_CONS_PERM` respectively).

Listing 1.3: The memory update function.

```
void update_memory() {
    update_count();
    update_hints();
    if (STRATEGY_TYPE == STRATEGY_CONSERVATIVE)
        update_conservative();
    if (STRATEGY_TYPE == STRATEGY_PERMISSIVE ||
        STRATEGY_TYPE == STRATEGY_CONS_PERM)
        update_permissive();
    if (HARDMODE)
        update_hard();
}

void update_count() {
    int local_counts[letter_t];
    for (p : pos_t)
        if (response[p] != GRAY)
            local_counts[guess[p]] += 1;
    for (c : letter_t)
        counts[c] = max(counts[c], local_counts[c]);
}

void update_conservative() {
    for (p : pos_t) {
        if (response[p] == GREEN) {
            for (l : letter_t)
                knowledge[p][l] = SURELY_NOT;
            knowledge[p][guess[p]] = SURELY;
        } else if (response[p] == YELLOW) {
            knowledge[p][guess[p]] = SURELY_NOT;
        } else if (response[p] == GRAY) {
            if (counts[guess[p]] == 0) { // absent
                for (q : pos_t)
                    knowledge[q][guess[p]] = SURELY_NOT;
            } else {
                knowledge[p][guess[p]] = SURELY_NOT;
                if (forall(q : pos_t)
                      guess[q] != guess[p] ||
                          response[q] != YELLOW)
                {   // all are correctly placed
                    for (q : pos_t)
                        if (guess[q] != guess[p] ||
                                response[q] != GREEN)
                            knowledge[q][guess[p]] = SURELY_NOT;
                }
```

```
            }
        }
    }
}

void update_permissive() {
    update_conservative();
    check_sums();
}

void check_sums() {
    int letter_sum;
    letter_t letter;
    int local_counts[letter_t];
    bool changed = true;
    while (changed) {
        changed = false;
        local_counts = counts;
        // check if only one unknown letter left
        for (p : pos_t) {
            letter_sum = 0;
            for (l : letter_t)
                if (knowledge[p][l] != SURELY_NOT) {
                    letter = l;
                    letter_sum += 1;
                }
            // new information?
            if (letter_sum = 1 &&
                    knowledge[p][letter] != SURELY) {
                knowledge[p][letter] = SURELY;
                changed = true;
            }
            if (letter_sum == 1)
                --local_counts[letter];
        }
        if ((sum(l : letter_t) counts[l]) == NPOS)
            // remove all inconsistent with the 5 known letters
            for (p : pos_t)
                for (l : letter_t)
                    if (knowledge[p][l] != SURELY &&
                            counts[l] == 0)
                        knowledge[p][l] = SURELY_NOT;
    }
}
```

Let us discuss the functions in order:

`update_count()` updates the count-array by first computing the sum of occurrences of letters in the guessed word for which the feedback was either green or yellow. In the end, this is aggregated into the `count` array by keeping only the largest value; the target word will not change during a single execution.

`update_conservative()` has three cases for each letter/response position:

1. if the response is `GREEN`, update the row corresponding to the position to `SURELY_NOT` and then mark the correct `GREEN` letter with the value `SURELY`,
2. if the response is `YELLOW`, update the single position/letter entry to the value `SURELY_NOT`, and
3. if the response is `GRAY`, either the target word contains no such letter (first sub-case) and we can update across the knowledge rows with the value `SURELY_NOT`. Alternatively we check whether, in the current guess, all other occurrences of the letter are correctly placed – in which case we can achieve a similar effect, and otherwise we limit ourselves to updating the single cell of the position/letter in the knowledge to `SURELY_NOT`.

`update_permissive()` calls first the `update_conservative()` function and then attempts to deduce even more values via a call to `check_sums()`.

`check_sums()` checks, if at a given position, only a single letter is possible, and then updates the knowledge array accordingly. If all 5 letters are known (but their positions not), the function will furthermore remove all non-valid letters from the knowledge array by marking them with the `SURELY_NOT` value.

We can now introduce the action-filtering code behind the different modes in Listing 1.4.

Listing 1.4: Game-mode filtering of actions.

```
bool mode_filter(wordid_t solution) {
    if(HARDMODE && !hard[solution]) return false;
    if (STRATEGY_TYPE == STRATEGY_CONSERVATIVE)
        return conservative_strategy(solution);
    else if (STRATEGY_TYPE == STRATEGY_PERMISSIVE)
        return permissive_strategy(solution);
    else if(STRATEGY_TYPE == STRATEGY_CONS_PERM)
        return combined_strategy(solution);
    else return true;
}

bool is_hard(wordid_t solution) {
    int local_counts[letter_t];
    bool ok_green;
    bool used_hints;
    ok_green = forall(p : pos_t)
                    knowledge[p][words[solution][p]] == SURELY ||
                    (not exists(l : letter_t)
                        knowledge[p][l] == SURELY);
```

```
20      local_counts = counts;
21      for(p : pos_t) --local_counts[words[solution][p]];
22      used_hints = forall(p : pos_t) local_counts[p] <= 0;
23      return ok_green && used_hints;
24  }
25
26  bool conservative_strategy(wordid_t solution) {
27      return not exists(p : pos_t)
        knowledge[p][words[solution][p]] == SURELY_NOT;
28  }
29
30  bool permissive_strategy(wordid_t solution) {
31      if (forall(p : pos_t)
32              knowledge[p][words[solution][p]] == SURELY)
33          return true;
34      // anything that feasibly improves the knowledge is allowed
35      return exists(p : pos_t)
36          knowledge[p][words[solution][p]] == MAYBE;
37  }
38
39  bool combined_strategy(wordid_t solution) {
40      if (forall(p : pos_t)
41              knowledge[p][words[solution][p]] == SURELY)
42          return true;
43      // anything that feasibly improves the knowledge is allowed
44      // also must not use characters shown to be useless
45      return exists(p : pos_t)
46          knowledge[p][words[solution][p]] == MAYBE &&
47             forall(p : pos_t)
48                 knowledge[p][words[solution][p]] != SURELY_NOT;
49  }
```

The `mode_filter(s)` function checks if the word with index `s` is valid guess based on the game mode – `HARDMODE` or particular `STRATEGY_TYPE`, which effectively filters the actions of the Player based on the different heuristics described.

Hard mode checks firstly that if `GREEN` hint was ever given, then the proposed word *must* contain the said letter in the said position. Secondly we validate that the sum of occurrences of letters matches with `YELLOW` and `GREEN` hints given prior.

Conservative mode is instead concerned with `GRAY` (absence) hints and strictly disallows any word where `GRAY` hint has been given. This generalizes to disallowing any word where the value of the knowledge for any letter of the word is `SURELY_NOT` (must be absent).

Permissive mode instead attempts to be as liberal as possible, but ensuring the progress. The mode disallows any word (except for the solution-word), which will not yield an update to the `knowledge`-array. This is done by requiring at least the presence of a single `MAYBE` in the knowledge vector, for a letter/position-combination of the proposed word.

Furthermore, we conjecture the following:

- the projection state of WORDLE on the POMDP with the observable state of the `knowledge` and `counts` arrays is sufficient to compute optimal strategies, and
- the optimal solution to the permissive game mode is also optimal in the original WORDLE.

Here optimality should be understood as the expected number of guesses, assuming that the Opponent uses a uniform random choice to select the solution word.

Partial Observable WORDLE Similar to the Monty Hall Problem presented in Section 3, we employ partial observability to correctly capture the nature of WORDLE. To follow our conjecture, we assume that it is sufficient for the strategy to depend on the `knowledge` and `count` arrays as observable entities. We thus construct the following query to synthesize near-optimal strategies:

```
strategy S = minE(guesses) [<=MAXG] {
    Play.counts[0], ..., Play.counts[25],
    Play.knowledge[0][0], ..., Play.knowledge[0][25],
    ...
    Play.knowledge[4][0], ..., Play.knowledge[4][25]
} -> {}: <> Play.Goal
```

Here `MAXG` is a sufficiently high bound on time, i.e., the number of guesses within which the game is expected to be completed. For all practical purposes and the proposed game modes, a value of 100 suffices, as witnessed in the experiments (See Figure 8).

However, 6×26 memory variables can take three different values ($\{0, 1, 2\}$ for `counts`, `SURELY_NOT`, `MAYBE` and `SURELY` for the `knowledge`), the hypothetical number of combinations is astronomically high and cannot be visited exhaustively. Therefore we also experiment with the partition refinement technique of UPPAAL STRATEGO to automatically deduce a sufficient partitioning of the state-space. This is reflected in the following query:

```
strategy S = minE(guesses) [<=MAXG] {
    Play.counts[0], ..., Play.counts[25]
} -> {
    Play.knowledge[0][0], ..., Play.knowledge[0][25],
    ...
    Play.knowledge[4][0], ..., Play.knowledge[4][25]
}: <> Play.Goal
```

Here we retain that each unique assignment to `counts` must lead to a different partition, i.e. a pre-partitioning of the observed state-space, while assignment to the `knowledge`-array can be aggregated at will of the learning-algorithm.

Informally, the learning algorithms *can* chose to lump together different assignments of the `knowledge` array, but *must* consider assignments of `counts` as being radically different.

The choice of placement of these observable variables is not arbitrary. The expected actual assignments to the `counts` array is expected to be small, and on the other hand, different assignments to this array is expected to have very different optimal strategies; i.e. best next word. On the other hand, the `knowledge` array is expected to have a huge domain of possible assignments where minor differences in assignment will have little effect on the optimal next word. Again we refer the reader to the work of M. Jaeger et. al. [19] for the details of the partition refinement procedure.

5 Evaluation

We experiment with using UPPAAL STRATEGO as a solver for the POMDP induced by the encoding of the WORDLE game presented in the previous section and we study the impact of:

1. increasing the training budget,
2. the different game modes, and
3. the partition-refinement technique of UPPAAL STRATEGO (denoted PT).

As the game modes presented in Section 4 can be combined, we experiment not only with the individual game-modes but also combinations. In particular we consider as basis *CONS*, the conservative mode, *PERM*, the permissive mode, *COMB* the combined mode of *CONS* and *PERM*. These three game-modes are then further combined with the hard-mode (denoted *HM*). For each of such a combination we denote by the *baseline* the random strategy under the given game mode. That is, in a given state, the strategy picks uniformly among all actions (words) allowed by the game mode. We refrain from experimenting with *HM* directly as experiments demonstrated a low probability of winning the game under the baseline strategy within a reasonable number of guesses.

All experiments are conducted on dis-homogeneous hardware and we thus refrain from commenting explicitly on running-times. We note that 1000 simulations on an Intel i7-1165G7 takes roughly 90 seconds, and that training times can be extrapolated to within a factor of two from this measure. Furthermore, to reduce the simulation time, we have modeled various caching techniques and in addition utilize the external C-library linking to offload the updates of the `knowledge` and `count` arrays.

Evaluation: We experiment with the learning-algorithms in UPPAAL STRATEGO over the 6 different strategy profiles and for each profile use either the explicit observation vectors or the partition-refinement technique (PT). Specifically we

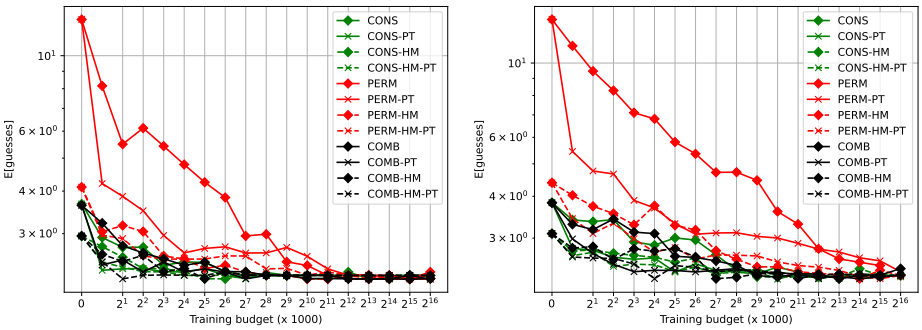
here also study the impact of increasing the training budget and we configure UPPAAL STRATEGO to utilize the full learning-budget and restrict the tool to only evaluate the quality of the strategy when the learning budget is exhausted. To evaluate the quality of a strategy `S`, we execute the query:

```
E[#<=MAXS;23150](max:guesses) under S
```

Notice that this evaluation allows for each word in the selected set of curated word set to be picked (on average) 10 times. Each experimental configuration is repeated 5 times and we report the best solution found for each configuration. We conduct three series of experiments; one with the full word-set of WORDLE and one where it is reduced by a factor of 64 and 128 denoted as WORDLE/64 and WORDLE/128 respectively. Let us first study the reduced versions of WORDLE.

WORDLE/**{64,128}:** In Figure 6a we see the gradual improvement in performance of the learning algorithm of UPPAAL STRATEGO when playing the WORDLE/128 game (reduced by a factor 128). We observe that all methods converge to strategies delivering on average 2.21 guesses. In the case of *COMB-HM* (black, dashed), the reduction from the baseline of 2.95 is modest while the game mode *PERM* (red, solid) has the largest improvement, having a baseline of 12.78 guesses. We can observe that the larger search-space of the permissive game mode (red) has a negative impact on the needed budget for convergence and without the game mode providing any benefit compared to the more restrictive game modes. In general all the tested configurations have converged with a budget of 4.096 million episodes.

We observe similar tendencies in WORDLE/64 (Figure 6b) with all modes converging on a performance of 2.27 guesses, however with the permissive game (red lines) modes not reaching a plateau and thus still with potential to improve beyond this limit. The *PERM* game mode (red, solid) here enjoys a drop from a baseline of 13.49 to 2.38 but also under the *COMB-HM* game mode the result is improved from a baseline of 3.94 to 2.27. Furthermore we see the partition refinement algorithm (annotated by x in the plot) generally appears to improve on the rate of learning initially, but is eventually, after enough episodes, overtaken by an explicit representation of the states (annotated by box in the plot). This indicates that the learning algorithm initially can exploit grouping large sets of states together under the same sub-strategy, and thus accumulate more information for this aggregate sub-strategy per episode. However, with enough samples, the partition-refinement scheme is overtaken when this same aggregation needs to be sub-partitioned, at which point the explicit representation has been trained sufficiently. Comparing Fig. 6a and Fig. 6b we can see that at least a 16 times larger training set is needed (2^4) for the learning methods to reach a plateau, indicating a radical growth in the total search space. Studying both plots we can also observe that an exponential increase in the learning budget (x-axis is in $\log_2$) leads to a less than linear improvement in the performance (y-axis is in $\log_{10}$). We hypothesize that this is due to the large action and sam-

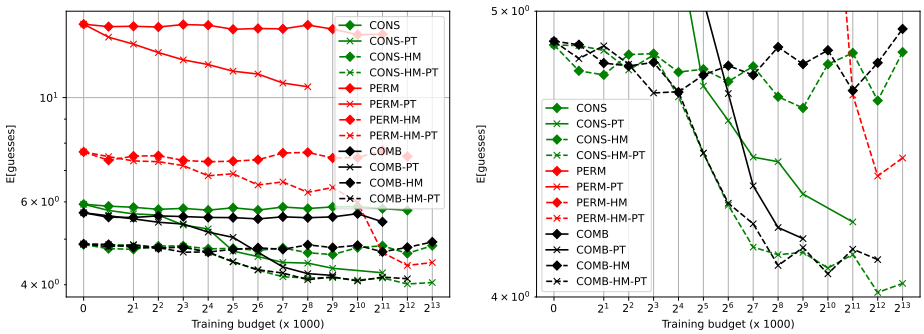


(a) WORDLE/128 (b) WORDLE/64

Fig. 6: Improvement of training using UPPAAL STRATEGO under different observability patterns and different strategies for the WORDLE model reduced by a factor of 128 (left) or 64 (right). The y-axis is given in $\log_{10}$ and the x-axis in $\log_2$. The 0-point on the x-axis represents the baseline (a training budget of 0 episodes).

ple space combined with the dampening of the updates of the Q-values when applying Q-learning.

Full WORDLE**:** As witnessed by Fig. 7a, UPPAAL STRATEGO is challenged more by the full version of WORDLE, however the tool still manages to produce a strategy yielding 4.01 guesses on average under the *COMB-HM* (black, dashed). This is a significant improvement compared to the 4.8 guesses on average when comparing to playing in the same game mode but using the baseline strategy. Given the astronomically large search space of WORDLE, having more than 20 million configurations after the first guess (2,315 possible "words of the day" times 12,972 possible initial guesses), finding an optimal solution is not expected. Similar to the reduced WORDLE games the *PERM* game mode (red) enjoys large improvements, dropping from a baseline of 7.67 guesses on average to 4.40 under hard-mode (red, dashed) and from 14.33 to 10.55 without hard-mode (red, solid). Comparing Fig. 7a and Fig. 7b where y-axis interval of $[4, 5]$ is in focus, it becomes apparent that a plateau has not yet been reached. We can observe that the best performing game modes slowly creep in an oscillating fashion further down, indicating that further episodes could further improve the performance. Contrary to what was observed in the WORDLE/128 and WORDLE/64 experiments, the explicit memory representation (annotated by squares) achieve little to no improvement when the number of training episodes is increased, although they still manage to outperform the random strategy. We can also see that the hard-mode (dashed lines) has a much higher impact. Comparing the *PERM-HM* (red, dashed) with *PERM* (red, solid) under the partition refinement (x annotated), we observe that the *PERM-HM* has a significantly lower starting-point and also achieves radical improvements around 1.024 million episodes. It remains



(a) Full WORDLE. (b) Zoomed to `guesses` $\in [4, 5]$.

Fig. 7: Improvement of training using UPPAAL STRATEGO under different observability patterns and different strategies for the full WORDLE game. The y-axis is given in $\log_{10}$ and the x-axis in $\log_2$. The 0-point on the x-axis represents the baseline (a training budget of 0 episodes).

to be seen whether the *PERM* exhibits the same behavior as the experiments have yet to complete at the time of writing.

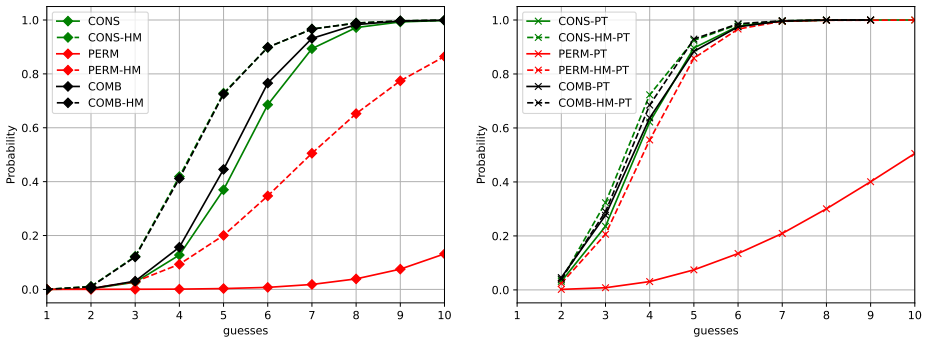
In Fig. 8 we can see that the different game-modes presented are effected vastly differently by training in terms of reduction in the expected number of guesses. Using the permissive game-mode yields a performance improvement from 14.33 guesses on average to 7.67 when hard mode is enabled as well. Playing according to the conservative mode leads to 5.93 guesses on average (4.87 under hard-mode) while the combined game-mode lands on average of 5.69 guesses (4.88 under hard-mode). While these game-modes overall reduce the number of guesses, none of them guarantee winning nor do they provide an impressive performance.

6 Conclusion

In this paper we have modelled and solved the popular game WORDLE using the tool UPPAAL STRATEGO constituting by orders of magnitude the largest

Table 1: Quality of the best obtained strategy for full WORDLE using partitioning (PT) or explicit memory representation in different modes. The Baseline row indicate the performance under a random strategy.

Game Mode	**CONS**	**CONS-HM**	**PERM**	**PERM-HM**	**COMB**	**COMB-HM**
Baseline	5.93	4.87	14.33	7.67	5.69	4.88
Explicit	5.76	4.64	13.60	7.30	5.44	4.69
PT	4.24	4.01	10.55	4.40	4.19	4.07



(a) Distribution before training.

(b) Distribution after training.

Fig. 8: Comparison of random plays under the permissive (red), conservative (green) and combined (black) strategies. The dashed lines indicate hard-mode enabled where solid lines indicate normal mode.

POMDP ever presented to the tool. The experimental evaluation are encouraging: our partitioning-refinement Q-learning method reduces significantly the expected number of guesses, e.g. from 7.67 to 4.40 guesses for the hard game-mode. In all our experiments, the partitioning-refinement method performs better than traditional, explicit Q-learning. Future research include learning directly from the feedback-history rather than from observation of the manually maintained `knowledge` and `count` data structures.

References

1. R. Alur and D. L. Dill. A theory of timed automata. *Theor. Comput. Sci.*, 126(2):183–235, 1994.
2. B. J. Anderson and J. G. Meyer. Finding the optimal human strategy for wordle using maximum correct letter probabilities and reinforcement learning, 2022.
3. G. Behrmann, A. Fehnker, T. Hune, K. G. Larsen, P. Pettersson, J. Romijn, and F. W. Vaandrager. Minimum-cost reachability for priced timed automata. In M. D. D. Benedetto and A. L. Sangiovanni-Vincentelli, editors, *Hybrid Systems: Computation and Control, 4th International Workshop, HSCC 2001, Rome, Italy, March 28-30, 2001, Proceedings*, volume 2034 of *Lecture Notes in Computer Science*, pages 147–161. Springer, 2001.
4. G. Behrmann, T. Hune, and F. W. Vaandrager. Distributing timed model checking - how the search order matters. In E. A. Emerson and A. P. Sistla, editors, *Computer Aided Verification, 12th International Conference, CAV 2000, Chicago, IL, USA, July 15-19, 2000, Proceedings*, volume 1855 of *Lecture Notes in Computer Science*, pages 216–231. Springer, 2000.
5. J. Bengtsson, W. O. D. Griffioen, K. J. Kristoffersen, K. G. Larsen, F. Larsson, P. Pettersson, and W. Yi. Verification of an audio protocol with bus collision using UPPAAL. In R. Alur and T. A. Henzinger, editors, *Computer Aided Verification, 8th International Conference, CAV '96, New Brunswick, NJ, USA, July 31 -*

August 3, 1996, Proceedings, volume 1102 of *Lecture Notes in Computer Science*, pages 244–256. Springer, 1996.

6. D. Bosscher, I. Polak, and F. W. Vaandrager. Verification of an audio control protocol. In H. Langmaack, W. P. de Roever, and J. Vytopil, editors, *Formal Techniques in Real-Time and Fault-Tolerant Systems, Third International Symposium Organized Jointly with the Working Group Provably Correct Systems - ProCoS, Lübeck, Germany, September 19-23, Proceedings*, volume 863 of *Lecture Notes in Computer Science*, pages 170–192. Springer, 1994.
7. K. A. Brown. Model, guess, check: Wordle as a primer on active learning for materials research. *npj Comput Mater*, 8(97), 5 2022.
8. A. David, P. G. Jensen, K. G. Larsen, M. Mikučionis, and J. H. Taankvist. Uppaal stratego. In C. Baier and C. Tinelli, editors, *Tools and Algorithms for the Construction and Analysis of Systems - 21st International Conference, TACAS 2015, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2015, London, UK, April 11-18, 2015. Proceedings*, volume 9035 of *Lecture Notes in Computer Science*, pages 206–211. Springer, 2015.
9. A. B. Downey. *The little book of semaphores (2nd edition): The ins and outs of concurrency control and common mistakes*. Createspace, 2009.
10. A. Eriksen, C. Huang, J. Kildebogaard, H. Lahrmann, K. Larsen, M. Muniz, and J. Taankvist. Uppaal stratego for intelligent traffic lights. In *12th ITS European Congress*. ERTICO - ITS Europe, 2017. 12th ITS European Congress : ITS Beyond Borders ; Conference date: 19-06-2017 Through 22-06-2017.
11. B. Gebremichael, F. W. Vaandrager, and M. Zhang. Analysis of the zeroconf protocol using UPPAAL. In S. L. Min and W. Yi, editors, *Proceedings of the 6th ACM & IEEE International conference on Embedded software, EMSOFT 2006, October 22-25, 2006, Seoul, Korea*, pages 242–251. ACM, 2006.
12. M. A. Goorden, K. G. Larsen, J. E. Nielsen, T. D. Nielsen, M. R. Rasmussen, and J. Srba. Learning safe and optimal control strategies for storm water detention ponds. In R. M. Jungers, N. Ozay, and A. Abate, editors, *7th IFAC Conference on Analysis and Design of Hybrid Systems, ADHS 2021, Brussels, Belgium, July 7-9, 2021*, volume 54 of *IFAC-PapersOnLine*, pages 13–18. Elsevier, 2021.
13. M. Hendriks, G. Behrmann, K. G. Larsen, P. Niebert, and F. W. Vaandrager. Adding symmetry reduction to uppaal. In K. G. Larsen and P. Niebert, editors, *Formal Modeling and Analysis of Timed Systems: First International Workshop, FORMATS 2003, Marseille, France, September 6-7, 2003. Revised Papers*, volume 2791 of *Lecture Notes in Computer Science*, pages 46–59. Springer, 2003.
14. M. Hendriks, B. van den Nieuwelaar, and F. W. Vaandrager. Model checker aided design of a controller for a wafer scanner. *Int. J. Softw. Tools Technol. Transf.*, 8(6):633–647, 2006.
15. T. A. Henzinger. The theory of hybrid automata. In *Proceedings, 11th Annual IEEE Symposium on Logic in Computer Science, New Brunswick, New Jersey, USA, July 27-30, 1996*, pages 278–292. IEEE Computer Society, 1996.
16. T. Hune, J. Romijn, M. Stoelinga, and F. W. Vaandrager. Linear parametric model checking of timed automata. In T. Margaria and W. Yi, editors, *Tools and Algorithms for the Construction and Analysis of Systems, 7th International Conference, TACAS 2001 Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2001 Genova, Italy, April 2-6, 2001, Proceedings*, volume 2031 of *Lecture Notes in Computer Science*, pages 189–203. Springer, 2001.

17. G. Igna, V. Kannan, Y. Yang, T. Basten, M. Geilen, F. W. Vaandrager, M. Voorhoeve, S. de Smet, and L. J. Somers. Formal modeling and scheduling of datapaths of digital document printers. In F. Cassez and C. Jard, editors, *Formal Modeling and Analysis of Timed Systems, 6th International Conference, FORMATS 2008, Saint Malo, France, September 15-17, 2008. Proceedings*, volume 5215 of *Lecture Notes in Computer Science*, pages 170–187. Springer, 2008.
18. G. Igna and F. Vaandrager. Verification of printer datapaths using timed automata. In T. Margaria and B. Steffen, editors, *Leveraging Applications of Formal Methods, Verification, and Validation*, pages 412–423, Berlin, Heidelberg, 2010. Springer Berlin Heidelberg.
19. M. Jaeger, P. G. Jensen, K. G. Larsen, A. Legay, S. Sedwards, and J. H. Taankvist. Teaching stratego to play ball: Optimal synthesis for continuous space mdps. In Y. Chen, C. Cheng, and J. Esparza, editors, *Automated Technology for Verification and Analysis - 17th International Symposium, ATVA 2019, Taipei, Taiwan, October 28-31, 2019, Proceedings*, volume 11781 of *Lecture Notes in Computer Science*, pages 81–97. Springer, 2019.
20. P. G. Jensen, K. G. Larsen, and J. Srba. Real-time strategy synthesis for timed-arc petri net games via discretization. In D. Bosnacki and A. Wijs, editors, *Model Checking Software - 23rd International Symposium, SPIN 2016, Co-located with ETAPS 2016, Eindhoven, The Netherlands, April 7-8, 2016, Proceedings*, volume 9641 of *Lecture Notes in Computer Science*, pages 129–146. Springer, 2016.
21. S. L. Karra, K. G. Larsen, F. Lorber, and J. Srba. Safe and time-optimal control for railway games. In S. C. Dutilleul, T. Lecomte, and A. B. Romanovsky, editors, *Reliability, Safety, and Security of Railway Systems. Modelling, Analysis, Verification, and Certification - Third International Conference, RSSRail 2019, Lille, France, June 4-6, 2019, Proceedings*, volume 11495 of *Lecture Notes in Computer Science*, pages 106–122. Springer, 2019.
22. K. G. Larsen, M. Mikučionis, and J. H. Taankvist. Safe and optimal adaptive cruise control. In R. Meyer, A. Platzer, and H. Wehrheim, editors, *Correct System Design - Symposium in Honor of Ernst-Rüdiger Olderog on the Occasion of His 60th Birthday, Oldenburg, Germany, September 8-9, 2015. Proceedings*, volume 9360 of *Lecture Notes in Computer Science*, pages 260–277. Springer, 2015.
23. D. Lokshtanov and B. Subercaseaux. Wordle is np-hard, 2022.
24. W. Rosenbaum. Finding a winning strategy for wordle is np-complete, 2022.
25. M. Schuts, F. Zhu, F. Heidarian, and F. W. Vaandrager. Modelling clock synchronization in the chess gmac WSN protocol. In S. Andova, A. McIver, P. R. D'Argenio, P. J. L. Cuijpers, J. Markovski, C. Morgan, and M. Núñez, editors, *Proceedings First Workshop on Quantitative Formal Methods: Theory and Applications, QFM 2009, Eindhoven, The Netherlands, 3rd November 2009*, volume 13 of *EPTCS*, pages 41–54, 2009.
26. F. Vaandrager. A first introduction to uppaal. *Deliverable no.: D5. 12 Title of Deliverable: Industrial Handbook*, 18, 2011.
27. F. W. Vaandrager and A. de Groot. Analysis of a biphase mark protocol with uppaaland PVS. *Formal Aspects Comput.*, 18(4):433–458, 2006.
28. Wikipedia. Wordle — Wikipedia, the free encyclopedia. `http://en.wikipedia.org/w/index.php?title=Wordle&oldid=1093753215`, 2022. [Online; accessed 19-June-2022].